

30/8/26 DAY 7:

Enumeration

enum

- enum is a user-defined data type in C++ that consists of integral constants, and each of them is given a name.
- It is used to assign names to the integral constants which makes a program easy to read and maintain.
- It is used to define a set of named integral constants that can be used to represent a collection of related values.
- It can be implicitly converted to an integer type, which can lead to accidental misuse of enumerators.
- We can explicitly assign values to the enumerators, or let the compiler assign them automatically starting from 0.

Syntax:

```
enum enum_name { value1, value2, ..., valueN };
```

Example:

```
enum Color { RED, GREEN, BLUE };
```

enum class

- enum class is a strongly typed enumeration introduced in C++11.
- It provides better type safety and scoping compared to the traditional enum.
- Each enumerator in an enum class must be accessed with the scope resolution operator ::.
- It cannot be implicitly converted to an integer type, which helps prevent accidental misuse of enumerators.

Syntax:

```
enum class enum_name { value1, value2, ..., valueN };
```

Example:

```
enum class Color { RED, GREEN, BLUE };
```

```
#include <iostream>
using namespace std;

enum Day{
    // Monday = 1, Tuesday = 2, Wednesday = 3, Thursday = 4, Friday = 5, Saturday = 6, Sunday = 7
    //Default: 0,1,2,3,4,5,6,7
    Monday = 1, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday
};

enum codes{
    hi = 10, hey, hello = 20, yo, namaste = 30, hola
};

enum StatusCodes{
    NotModified = 304, OK = 200, NotFound = 404, InternalServerError = 500
};

//enum color{
//    Red, Blue, Orange, Black, White
//};
//
//enum TrafficSignal{
//    Red, Green, Yellow
//};

enum class color{
    Red, Blue, Orange, Black, White
};

enum class TrafficSignal{
    Red, Green, Yellow
};

int main(){

    color c = color::Red;

    Day day = Monday;

    // int col = color::Red;    //error: cannot convert 'color' to 'int'

    int col = static_cast<int>(color::Red); // static cast

    // cout << c << endl;    // not allowed :error for operator overloading<<

    cout << day << endl;
    cout << col << endl;
    return 0;
}
```

typedef

- typedef is a keyword in C++ that allows you to create an alias for an existing data type.
- It can be used to simplify complex type declarations, improve code readability, and make it easier to change the underlying type in the future.
- It is often used to create more meaningful names for data types, especially when dealing with pointers, structures, and function pointers.

Syntax:

```
typedef existing_type new_type_name;
```

Example:

```
typedef unsigned int uint;  
typedef unsigned long long ull;
```

It does not create a new type, but rather provides an alternative name for an existing type.

Type Aliases

- Type aliases are a feature in C++ that allows you to create an alternative name for an existing data type.
- They can be created using the `typedef` keyword or the `using` keyword (introduced in C++11).
- Type aliases can improve code readability and maintainability by providing more meaningful names for complex types.
- Type aliases can be used with built-in types (int, double, float, unsigned int, unsigned long long, etc.), user-defined types (structs, classes), and template types (e.g., `std::vector<int>`).

using type aliases

- The `using` keyword can be used to create type aliases in C++. It provides a more modern and flexible way to define type aliases compared to `typedef`.
- It works similarly like `typedef`.

Syntax:

```
using new_type_name = existing_type;
```

Example:

```
using uint = unsigned int;  
using ull = unsigned long long;
```

Resource Ownership Concept

Resource

- A resource is any entity that requires explicit management, such as memory, file handles, network sockets, or database connections.

Example:

- Dynamic Memory
- File Handles
- Network Sockets
- Database Connections
- Graphics Resources
- OS Api Resources

Ownership

- Ownership refers to the responsibility of managing the lifecycle of a resource, including its allocation, usage, and deallocation.
- The owner of a resource is responsible for ensuring that the resource is properly released when it is no longer needed, preventing resource leaks and undefined behavior.
- Ownership can be transferred between different parts of a program, but it is essential to maintain clear ownership semantics to avoid issues such as double deletion, dangling pointers, and memory leaks.

Resource Management

- Resource management is the process of acquiring, using, and releasing resources in a controlled and efficient manner.
- It involves techniques and strategies to ensure that resources are allocated and deallocated correctly, preventing resource leaks, dangling pointers, and other issues related to resource ownership.

Memory Leak

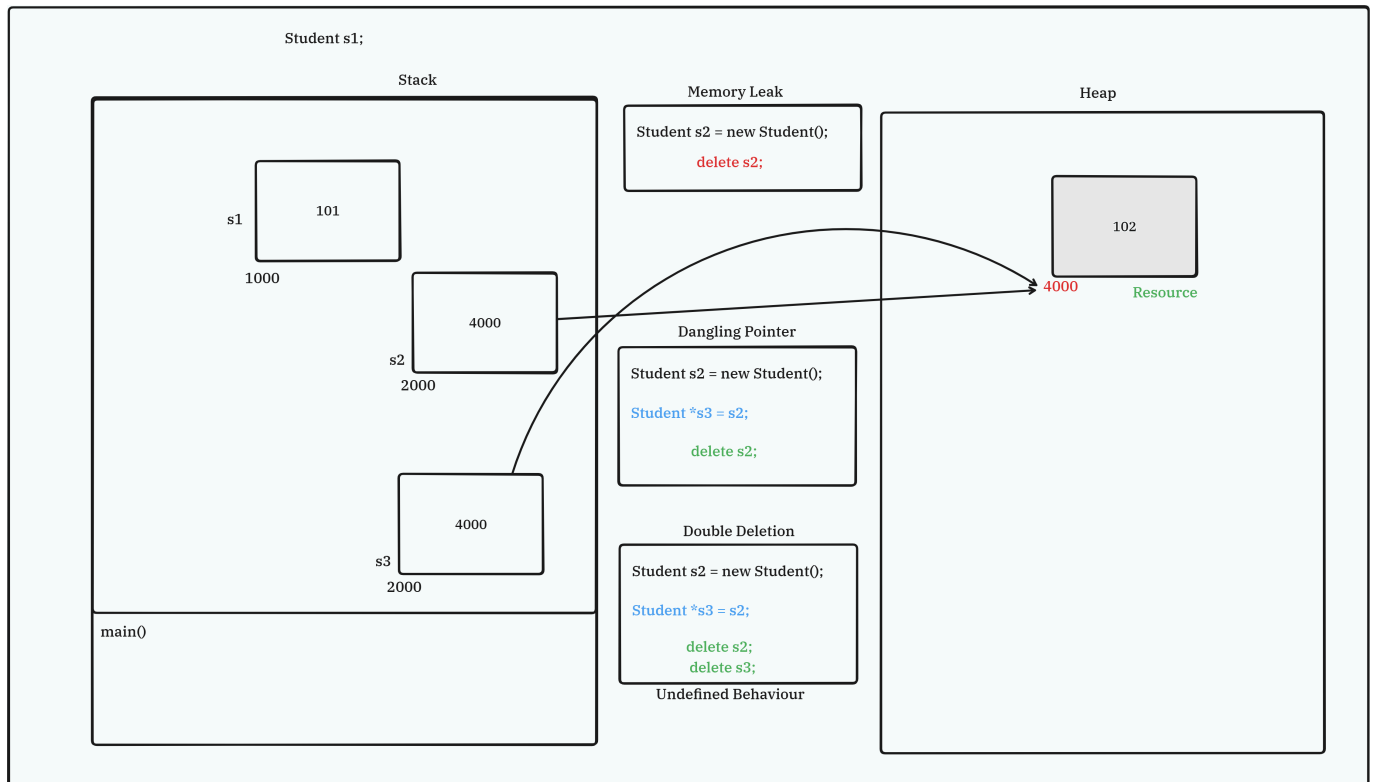
- Memory leak occurs when a program allocates memory but fails to release it back to the system, leading to a gradual increase in memory usage over time.
- This can result in performance degradation and, in severe cases, application crashes due to exhaustion of available memory.

Dangling Pointer

- A dangling pointer is a pointer that continues to reference a memory location after the object it points to has been deallocated or released.
- Accessing a dangling pointer can lead to undefined behavior, including crashes, data corruption, and security vulnerabilities.
- It is essential to ensure that pointers are properly managed and set to nullptr after the associated resource has been released.

Double Deletion

- Double deletion occurs when a program attempts to deallocate the same memory resource more than once, leading to undefined behavior and potential program crashes.
- This can happen when multiple pointers point to the same memory location, and one of them is deleted while the others still reference it.



RAII Concept

- **RAII (Resource Acquisition Is Initialization)** is a programming idiom used in C++ to manage resource ownership and ensure proper resource cleanup.
- The RAII concept ties the lifecycle of a resource to the lifetime of an object, ensuring that resources are acquired during object construction and released during object destruction.
- By using RAII, developers can avoid common resource management issues such as memory leaks, dangling pointers, and double deletion, as the resource management is automatically handled by the object's lifetime.

Introduction to Smart Pointers

- Smart pointers are a type of class in C++ that manage the lifetime of dynamically allocated objects. They provide automatic memory management and help prevent common issues such as memory leaks, dangling pointers, and double deletion.

Why Smart Pointers?

- Smart pointers are used to manage the ownership and lifetime of dynamically allocated objects in C++. They provide automatic memory management, preventing memory leaks and dangling pointers.
- Smart pointers help enforce ownership semantics, making it clear which part of the code is responsible for managing the resource. This reduces the risk of double deletion and other resource management issues.

Tomorrow's Topics:

- `unique_ptr`
- Single Ownership
- `make_unique()`
- Automatic Resource Management
- Moving Ownership
- `shared_ptr`
- Shared Ownership
- Reference Counting
- `make_shared()`
- `weak_ptr`
- Non-owning Reference
- `lock()`
- `expired()`
- Breaking Ownership Cycles
- Smart Pointer Comparison
- Raw Pointer vs Smart Pointer
- Smart Pointers with Objects
- Smart Pointers and Function Parameters